

Milestone 8 —Import catalogue & Export comptabilité CSV

Le support est une propriété de **Bechir Bejaoui**

Objectif du sprint

Connecter **AI Qalam Stock Manager** au monde extérieur via des fichiers CSV.
À la fin de ce sprint, l'application peut :

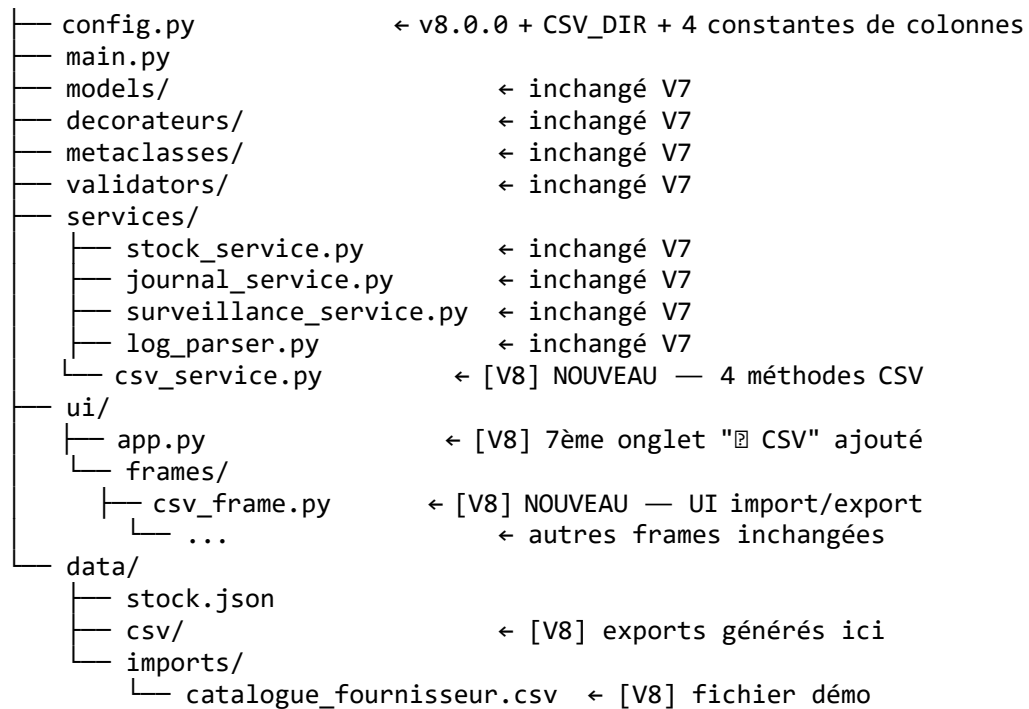
- **Importer** un catalogue fournisseur CSV → créer ou mettre à jour les produits
 - **Exporter** le catalogue complet → partager avec un partenaire ou sauvegarder
 - **Exporter les mouvements** → audit comptable, contrôle de gestion
 - **Exporter un rapport comptable** → valeur stock, marges unitaires et totales
-

Concepts Python mis en pratique

Concept	Module	Usage dans AI Qalam
csv.DictReader	csv	Lecture import par nom de colonne
csv.DictWriter	csv	Écriture exports structurés
encoding="utf-8-sig"	open()	Compatibilité Excel (BOM UTF-8)
newline=""	open()	Évite \r\r\n sous Windows
Validation fieldnames	csv.DictReader	Vérification colonnes avant traitement
try/except ValueError	Python	Conversions de types robustes
Path / filedialog	pathlib, tkinter	Parcours fichiers dans l'UI

Architecture V8

alqalamv8/



Changements V8 résumés : - config.py : version 8.0.0, CSV_DIR, 4 constantes de colonnes - services/csv_service.py : nouveau —CsvService + ErreurCsv - ui/frames/csv_frame.py : nouveau —onglet import/export avec prévisualisation - ui/app.py : 7ème onglet branché, polling arrêté proprement

1. Pourquoi csv et pas pandas ici ?

Module csv (standard)	pandas (externe)
☑ Intégré Python — aucun pip	☑ pip install pandas requis
☑ Mémoire constante (streaming)	☑ charge tout en RAM
☑ Parfait pour import/export	☑ Meilleur pour analyse/stats
☑ Contrôle total ligne par ligne	☑ Filtres/agrégations en 1 ligne
☑ Pas de dépendance externe	☑ Idéal pour > 10 000 lignes

Règle de décision pour Al Qalam : - Import fournisseur < 1 000 lignes → csv (léger, aucune dépendance) - Analyse de tendances > 10 000 mouvements → pandas (sprint futur)

2. config.py —Les constantes CSV (V8)

```
# — CSV V8 —————  
CSV_DIR = DATA_DIR / "csv"  # dossier dédié aux exports générés  
  
# Colonnes attendues dans un CSV fournisseur (import)  
COLONNES_CATALOGUE_IMPORT = ["ref", "nom", "categorie", "prix_achat", "prix_vente", "s  
  
# Colonnes exportées (include qte — donnée interne AL Qalam)  
COLONNES_CATALOGUE_EXPORT = ["ref", "nom", "categorie", "prix_achat", "prix_vente", "c  
  
# Colonnes du rapport de mouvements  
COLONNES_MOUVEMENTS = ["date", "type", "ref", "produit", "qte", "note"]  
  
# Colonnes du rapport comptable (valeurs calculées)  
COLONNES_COMPTABILITE = ["ref", "nom", "categorie", "qte", "prix_achat", "prix_vente",  
                          "valeur_stock", "marge_unitaire", "marge_totale"]  
  
Pourquoi des constantes ? - Un seul endroit à modifier si le format CSV change  
- Le code dans CsvService et CsvFrame reste stable - Testable: assert "ref"  
in COLONNES_CATALOGUE_IMPORT
```

3. services/csv_service.py —CsvService

3.1 Structure de la classe

```
class ErreurCsv(Exception):  
    """Levée quand un fichier CSV ne respecte pas la structure attendue."""  
  
class CsvService:  
    """  
    Service d'import/export CSV pour AL Qalam.  
  
    [V8] Utilise uniquement Le module standard `csv` :  
    - Aucune dépendance externe (pas de pandas)  
    - Traitement ligne par ligne → consommation mémoire constante  
    - Compatible Excel grâce à encoding="utf-8-sig" (BOM)  
  
    Usage :  
    service = CsvService(stock_service)  
    rapport = service.importer_catalogue("fournisseur.csv")  
    service.exporter_comptabilite("rapport_mars.csv")  
    """
```

```
def __init__(self, stock_service):
    self._stock = stock_service # injection de dépendance (pas un Singleton)
    CSV_DIR.mkdir(parents=True, exist_ok=True)
```

CsvService **n'est pas un Singleton** —il peut y avoir plusieurs instances (une par fenêtre de dialogue par exemple). Il **dépend** de StockService (Singleton V6).

3.2 Import —importer_catalogue()

```
@journaliser("import_csv")
def importer_catalogue(self, chemin: str | Path) -> dict:
    """
    Importe un catalogue fournisseur CSV dans le stock.

    Comportement :
    - ref absente du stock → ajouter_produit() avec qte=0
    - ref déjà présente → mettre_a_jour_produit() (prix + seuil)
    - ligne invalide → ajoutée à erreurs[], traitement continue

    Returns:
    dict {
        "importes" : nb de nouveaux produits créés,
        "mis_a_jour": nb de produits mis à jour,
        "erreurs" : liste de str décrivant les lignes rejetées,
        "total_lues": nb total de lignes traitées
    }
    """
    chemin = Path(chemin)
    if not chemin.exists():
        raise FileNotFoundError(f"Fichier introuvable : {chemin}")

    rapport = {"importes": 0, "mis_a_jour": 0, "erreurs": [], "total_lues": 0}

    with open(chemin, "r", encoding="utf-8-sig") as f:
        lecteur = csv.DictReader(f)

        # — 1. Valider la structure AVANT de traiter —————
        colonnes_presentes = set(lecteur.fieldnames or [])
        manquantes = set(COLONNES_CATALOGUE_IMPORT) - colonnes_presentes
        if manquantes:
            raise ErreurCsv(
                f"Colonnes manquantes dans {chemin.name} : "
                f"'{', '.join(sorted(manquantes))}'"
            )
```

```

# — 2. Traitement ligne par ligne —————
for numero_ligne, ligne in enumerate(lecteur, start=2):
    rapport["total_lues"] += 1
    try:
        ref          = ligne["ref"].strip()
        nom           = ligne["nom"].strip()
        categorie     = ligne["categorie"].strip()
        prix_achat    = float(ligne["prix_achat"].replace(",", "."))
        prix_vente    = float(ligne["prix_vente"].replace(",", "."))
        seuil_min     = int(ligne["seuil_min"])

        if not ref or not nom or not categorie:
            raise ValueError("ref, nom et categorie ne peuvent pas être vides")

        if prix_achat < 0 or prix_vente < 0 or seuil_min < 0:
            raise ValueError("Les valeurs numériques doivent être positives")

        if ref in self._stock:
            # Produit existant → mise à jour prix et seuil
            existant = self._stock.get_produit(ref)
            produit_maj = Produit(
                ref=existant.ref, nom=nom, categorie=categorie,
                prix_achat=prix_achat, prix_vente=prix_vente,
                qte=existant.qte, # ← on conserve la qte en stock !
                seuil_min=seuil_min,
            )
            self._stock.mettre_a_jour_produit(produit_maj)
            rapport["mis_a_jour"] += 1
        else:
            # Nouveau produit — qte=0 (à réceptionner via entrée stock)
            nouveau = Produit(
                ref=ref, nom=nom, categorie=categorie,
                prix_achat=prix_achat, prix_vente=prix_vente,
                qte=0, seuil_min=seuil_min,
            )
            self._stock.ajouter_produit(nouveau)
            rapport["importes"] += 1

    except (ValueError, KeyError) as e:
        rapport["erreurs"].append(f"Ligne {numero_ligne} rejetée : {e}")
        continue # une ligne invalide ne stoppe pas l'import

return rapport

```

Points clés :

1. encoding="utf-8-sig"

- Lit le BOM (Byte Order Mark) si présent (exports Excel)
 - Sans le "-sig", le BOM apparaît comme "\ufeffref" dans fieldnames
2. Validation fieldnames avant itération
 - Si colonnes manquantes : ErreurCsv immédiate (fail fast)
 - Évite KeyError silencieux au milieu de milliers de lignes
 3. ligne["prix_achat"].replace(",", ".")
 - CSV européens utilisent "," comme séparateur décimal
 - "2,50" → "2.50" → float(2.50) ☑
 4. ref in self._stock (utilise __contains__ de StockService V2)
 - import idempotent : relancer 2 fois le même CSV = même résultat
 5. qte=existant.qte lors d'une mise à jour
 - Le fournisseur ne connaît pas notre stock physique
 - On préserve la quantité réelle, on met à jour prix/seuil uniquement
 6. continue dans except
 - Une ligne invalide est logguée mais ne bloque pas les suivantes
 - Rapport d'erreurs complet à la fin

3.3 Export catalogue —exporter_catalogue()

```
@journaliser("export_catalogue_csv")
def exporter_catalogue(self, chemin: str | Path | None = None) -> int:
    if chemin is None:
        horodatage = datetime.now().strftime("%Y%m%d_%H%M%S")
        chemin = CSV_DIR / f"catalogue_{horodatage}.csv"

    chemin = Path(chemin)
    chemin.parent.mkdir(parents=True, exist_ok=True)

    produits = self._stock.lister_tous()

    with open(chemin, "w", newline="", encoding="utf-8-sig") as f:
        #                               ↑
        #          obligatoire Windows   BOM pour Excel
        writer = csv.DictWriter(f, fieldnames=COLONNES_CATALOGUE_EXPORT)
        writer.writeheader() # ← ne pas oublier !

        for p in sorted(produits): # tri alphabétique via __lt__ de Produit
            writer.writerow({
                "ref"       : p.ref,
                "nom"        : p.nom,
                "categorie"  : p.categorie,
```

```

        "prix_achat": p.prix_achat,
        "prix_vente": p.prix_vente,
        "qte"       : p.qte,
        "seuil_min" : p.seuil_min,
    })

```

```

    return len(produits)

```

newline="" —pourquoi c'est obligatoire sous Windows :

Sans newline="" :

Python ouvre en mode texte → traduit "\n" en "\r\n"

csv.writer ajoute aussi "\r\n" (standard RFC 4180)

Résultat : "\r\r\n" → ligne vide entre chaque ligne dans Excel

Avec newline="" :

Python ne traduit rien → csv.writer seul gère les fins de ligne

Résultat : "\r\n" correct → fichier propre dans Excel

3.4 Export mouvements —exporter_mouvements()

```

@journaliser("export_mouvements_csv")

```

```

def exporter_mouvements(self, chemin: str | Path | None = None) -> int:
    # ... (même pattern que exporter_catalogue) ...

```

```

    mouvements = self._stock.get_mouvements()

```

```

    with open(chemin, "w", newline="", encoding="utf-8-sig") as f:
        writer = csv.DictWriter(f, fieldnames=COLONNES_MOUVEMENTS)
        writer.writeheader()

```

```

    for m in mouvements:

```

```

        try:

```

```

            nom_produit = self._stock.get_produit(m.ref).nom

```

```

        except KeyError:

```

```

            nom_produit = m.ref # produit supprimé depuis → fallback sur ref

```

```

        writer.writerow({
            "date"    : m.date.strftime("%Y-%m-%d %H:%M:%S"),
            "type"    : m.type_mvt,
            "ref"     : m.ref,
            "produit" : nom_produit,
            "qte"     : m.qte,
            "note"    : m.note,
        })

```

```

    return len(mouvements)

```

Résultat dans Excel :

date	type	ref	produit	qte	note
2026-03-09 14:22:01	entree	CRAY-001	Crayon HB	50	
2026-03-09 14:23:15	sortie	STYL-001	Stylo Bleu	10	Vente client 42
2026-03-09 14:24:00	import_csv	---	---	0	catalogue_fournisseur.csv

3.5 Export comptable —exporter_comptabilite()

```
for p in sorted(produits):
    writer.writerow({
        "ref" : p.ref,
        "nom" : p.nom,
        "categorie" : p.categorie,
        "qte" : p.qte,
        "prix_achat" : f"{p.prix_achat:.3f}", # 3 décimales TND
        "prix_vente" : f"{p.prix_vente:.3f}",
        "valeur_stock" : f"{p.valeur_stock():.3f}", # qte × prix_achat
        "marge_unitaire" : f"{p.marge_unitaire():.3f}", # prix_vente - prix_achat
        "marge_totale" : f"{p.qte * p.marge_unitaire():.3f}", # qte × marge
    })
```

Extrait du CSV généré :

```
ref,nom,categorie,qte,prix_achat,prix_vente,valeur_stock,marge_unitaire,marge_totale
CAR-001,Carnet A5,Papier,3,1.200,3.500,3.600,2.300,6.900
CIS-001,Ciseaux 17cm,Coupe,25,1.500,4.000,37.500,2.500,62.500
CRAY-001,Crayon HB,Écriture,150,0.150,0.500,22.500,0.350,52.500
...
```

3.6 Prévisualisation —previsualiser_csv()

```
def previsualiser_csv(self, chemin: str | Path, nb_lignes: int = 5) -> dict:
    """Lit les premières lignes d'un CSV pour prévisualiser avant import."""
    chemin = Path(chemin)
    resultat = {"colonnes": [], "lignes": [], "total_col": 0, "valide": False}

    if not chemin.exists():
        return resultat

    with open(chemin, "r", encoding="utf-8-sig") as f:
        lecteur = csv.DictReader(f)
        colonnes = list(lecteur.fieldnames or [])
        resultat["colonnes"] = colonnes
        resultat["total_col"] = len(colonnes)

    # Vérifie si toutes les colonnes requises sont présentes
```



```

requises = set(COLONNES_CATALOGUE_IMPORT)
resultat["valide"] = requises.issubset(set(colonnes))

for i, ligne in enumerate(lecteur):
    if i >= nb_lignes:
        break # on lit seulement nb_lignes lignes
    resultat["lignes"].append(dict(ligne))

return resultat

```

Pourquoi prévisualiser avant d'importer ? - UX : l'utilisateur voit les données avant de valider - Validation visuelle : mauvais encodage ? colonnes manquantes ? - Pas de modification du stock pendant la prévisualisation

4. ui/frames/csv_frame.py —Interface import/export

4.1 Architecture de l'onglet

Gestion CSV — Import / Export	
☐ Import catalogue fournisseur [Chemin du fichier] [Parcourir] ref nom cat prix_a (5 lignes prévisualisation) ☐ Import terminé ! Importés : 10 Mis à jour : 2 Erreurs : 0	☐ Exports CSV ☐ Catalogue produits ☐ Historique mouvements ☐ Rapport comptable Fichiers exportés : [14:22] ☐ Catalogue : ... [14:23] ☐ Comptabilité : ... Stock courant : 20 produits 5 alerte(s)
☐ Importer dans le stock	

4.2 Flux d'import complet

Utilisateur clique "Parcourir"



filedialog.askopenfilename() → chemin sélectionné

```

↓
CsvService.previsualiser_csv(chemin, nb_lignes=5)
↓
_afficher_previsualisation(aperçu) → Treeview mis à jour
↓
aperçu["valide"] ?
    Oui → btn_importer activé (vert) + rapport "Structure valide"
    Non → btn_importer désactivé (rouge) + rapport "Colonnes manquantes"
↓
Utilisateur clique "Importer dans le stock"
↓
CsvService.importer_catalogue(chemin)
↓
rapport → affichage dans txt_rapport
↓
self.rafraichir() → compteur mis à jour
↓
messagebox.showinfo(résumé)

```

4.3 Gestion des erreurs UI

```

def _importer(self):
    try:
        rapport = self.csv_svc.importer_catalogue(self._chemin_import)
        # ... afficher le rapport ...

    except FileNotFoundError as e:
        # Fichier supprimé entre la prévisualisation et L'import
        messagebox.showerror("Fichier introuvable", str(e))

    except ErreurCsv as e:
        # Structure invalide détectée à L'import
        messagebox.showerror("Erreur structure CSV", str(e))
        self._ecrire_rapport(f"Erreur : {e}")

    except Exception as e:
        # Erreur inattendue (disque plein, droits, etc.)
        messagebox.showerror("Erreur", f"Erreur inattendue : \n{e}")

```

Hiérarchie des erreurs CSV :

```

Exception
├── FileNotFoundError    ← fichier absent
├── ErreurCsv           ← colonnes manquantes (notre exception custom)
├── ValueError          ← conversions de types échouées (dans CsvService)
└── Exception           ← cas non prévus (disque plein, permissions...)

```

5. ui/app.py — Les 4 changements V8

```
# — 1. Import —————
from ui.frames.csv_frame import CsvFrame # [V8]

# — 2. Ajout du 7ème onglet —————
self.tabs.add("CSV") # dans _construire_onglets()
self.csv_frame = CsvFrame(self.tabs.tab("CSV"), self.stock)
self.csv_frame.pack(fill="both", expand=True)

# — 3. Rafraîchissement post-opération —————
def _post_operation(self):
    """Rafraîchit les sept onglets après toute modification du stock."""
    self.stock_frame.rafraichir()
    self.rapport_frame.rafraichir()
    self.alertes_frame.rafraichir()
    self.journal_frame.rafraichir()
    self.registre_frame.rafraichir()
    self.analyseur_frame.rafraichir()
    self.csv_frame.rafraichir() # [V8]

# — 4. Arrêt propre du polling —————
def _quitter(self):
    # ...
    self.csv_frame.arreter_polling() # [V8]
```

6. Fichier CSV de démo fournisseur

data/imports/catalogue_fournisseur.csv :

```
ref,nom,categorie,prix_achat,prix_vente,seuil_min
CRAY-003,Crayon Couleur Rouge,Écriture,0.25,0.80,15
CRAY-004,Crayon Couleur Bleu,Écriture,0.25,0.80,15
STYL-003,Stylo Vert,Écriture,0.30,0.90,25
STYL-004,Stylo Noir,Écriture,0.30,0.90,25
GOM-002,Gomme Rose,Effaçage,0.18,0.60,10
PAP-B5,Rame Papier B5,Papier,3.00,6.50,20
CIS-002,Ciseaux 21cm,Coupe,2.00,5.50,5
REG-002,Règle 20cm,Mesure,0.60,1.80,10
TAM-001,Taille-crayon,Écriture,0.40,1.20,20
COL-001,Colle Bâton,Adhésif,0.80,2.50,15
CRAY-001,Crayon HB,Écriture,0.18,0.55,25
```

PAP-A4,Rame Papier A4,Papier,2.80,5.50,60

Scénarios testés par ce fichier : - Lignes 1–10 : 10 nouveaux produits → créés avec qte=0 - Lignes 11–12 : CRAY-001 et PAP-A4 déjà présents → **mise à jour** prix + seuil, qte conservée

Simulation du rapport d'import :

```
❏ Import terminé !
  Lignes lues      : 12
  Importés         : 10 nouveaux produits
  Mis à jour       : 2 produits existants
  Erreurs          : 0
```

7. Bonnes pratiques CSV —résumé opérationnel

```
import csv
from pathlib import Path

# ❏ 1. Toujours encoding="utf-8-sig" pour la compatibilité Excel
with open("fichier.csv", "r", encoding="utf-8-sig") as f:
    lecteur = csv.DictReader(f)

# ❏ 2. Toujours newline="" à l'écriture sous Windows
with open("sortie.csv", "w", newline="", encoding="utf-8-sig") as f:
    writer = csv.DictWriter(f, fieldnames=["ref", "nom"])
    writer.writeheader() # ← ne jamais oublier !

# ❏ 3. Valider fieldnames AVANT d'itérer
colonnes_requises = {"ref", "nom", "categorie", "prix_achat"}
manquantes = colonnes_requises - set(lecteur.fieldnames or [])
if manquantes:
    raise ErreurCsv(f"Colonnes manquantes : {manquantes}")

# ❏ 4. DictReader pour l'accès par nom (résistant aux réordonnements)
for ligne in lecteur:
    ref = ligne["ref"] # ❏ stable
    # ref = ligne[0]   # ❏ fragile si ordre change

# ❏ 5. Convertir dans un try/except, jamais directement
try:
    prix = float(ligne["prix_achat"].replace(",", "."))
    seuil = int(ligne["seuil_min"])
except ValueError as e:
    erreurs.append(f"Ligne {num} : {e}")
```

```

        continue # ← on continue, pas de plantage global

# 6. strip() sur les champs texte
ref = ligne["ref"].strip() # " CRAY-001 " → "CRAY-001"

# 7. Chemin par défaut avec horodatage si non fourni
if chemin is None:
    chemin = CSV_DIR / f"export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"

# 8. mkdir(parents=True, exist_ok=True) avant d'écrire
Path(chemin).parent.mkdir(parents=True, exist_ok=True)

```

8. Démarrer V8

Depuis Le dossier alqalamv8
 py -3 main.py

Scénario de démonstration :

1. Lancer l'application → onglet "CSV"
 2. Cliquer "Parcourir" → sélectionner data/imports/catalogue_fournisseur.csv
 3. Observer la prévisualisation (5 premières lignes dans le Treeview)
 4. Vérifier le message "Structure valide"
 5. Cliquer "Importer dans le stock" → rapport : 10 importés, 2 mis à jour
 6. Aller dans l'onglet "Stock" → les 10 nouveaux produits sont visibles avec qte=0
 7. Revenir dans "CSV" → cliquer "Catalogue produits" → sauvegarder → ouvrir dans Excel
 8. Cliquer "Rapport comptable" → vérifier les colonnes valeur_stock, marge_unitaire, marge_totale
 9. Effectuer quelques entrées stock pour les nouveaux produits
 10. Cliquer "Historique mouvements" → vérifier que l'import est tracé dans le CSV
-

9. Ce que ce sprint apporte à la progression

V1	P00 de base	→	Produit, StockService, JSON
V2	Méthodes magiques	→	__lt__, __contains__, tri, contextes
V3	Compréhensions	→	filtres, top 5, stats catégories
V4	Threading	→	surveillance arrière-plan, Lock, Queue
V5	Décorateurs	→	@journaliser, @valider_qte, descripteurs
V6	Métaclasses	→	Singleton, Registre mouvements
V7	Regex	→	validators temps réel, log parser

V8 CSV → import/export, DictReader/Writer, BOM, filedialog
V9 Excel (suivant) → openpyxl, rapports colorés, multi-feuilles
V10 SQLite → persistance complète, historique requêtes
V11 Tests → pytest, fixtures, couverture 80%

Ce que V8 introduit de nouveau : - Premier contact avec l'**acquisition de données externes** (fichiers tiers) - Pattern **import idempotent** : même résultat si on relance (pas de doublons) - **Exception custom** (ErreurCsv) pour distinguer erreurs métier vs système - **Prévisualisation avant action** : principe UX important - **Rapport de traitement** retourné par la méthode (pas juste un bool)

□ Récapitulatif V8

CsvService utilise `csv.DictReader` (import) et `csv.DictWriter` (export) avec `encoding="utf-8-sig"` pour Excel et `newline=""` pour Windows. La validation des `fieldnames` avant traitement garantit un **fail fast** propre. L'import est **idempotent** (relancer = même résultat), conserve la qte physique lors des mises à jour, et retourne un **rapport détaillé** plutôt qu'un simple booléen. L'UI offre une **prévisualisation** avant import et un **journal** des exports générés.